Insert After An Element

And

TIme Complexity

Program:

```
void insertAfterElement(Node **head, int data, int after)
{
    Node *ptr, *loc;
    ptr = (Node *)malloc(sizeof(Node));
    loc = searchInUnsortedList(head, after);
    if (loc == nullptr)
    {
        cout << "Error: Element not found." << endl;
        return;
    }
    ptr->data = data;
    ptr->next = loc->next;
    loc->next = ptr;
}

…..

case 5:
        {
            int data, after;
            cout << "Enter the data: ";
            cin >> data;
            cout << "Enter the element after which to insert: ";
            cin >> after;
            insertAfterElement(&head, data, after);
            break;
        }
```

Program Analysis:

```
    Node *ptr;
    ptr = (Node *)malloc(sizeof(Node));
```

→ *The size of ptr will be equal to the size of a Node structure and ptr is declared as a pointer to Node structure.*

→ *The size of a pointer (ptr) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

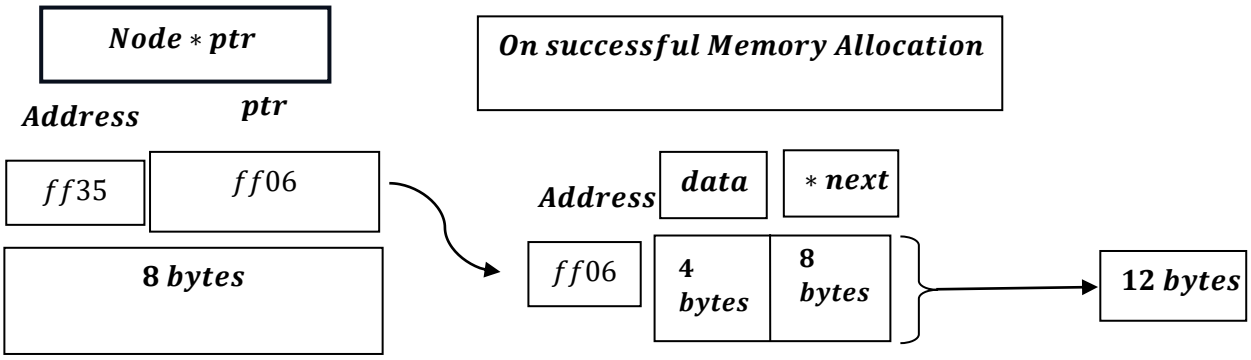
→ *data(an integer) is typically 4 bytes.*

→ *next(a pointer to another Node) is typically 8 bytes.*

→ *Therefore, the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for next).*

→ *ptr will try to allocate memory for data and next pointer variable i.e. typically 8 bytes, though for variable data and structure Node's pointer variable next in total 12 bytes, but as discussed earlier, the size of a pointer (ptr) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.*

Say,



Now, let's , see three condition's in unsorted search to apply on insert after an element.

1) *List is empty*:

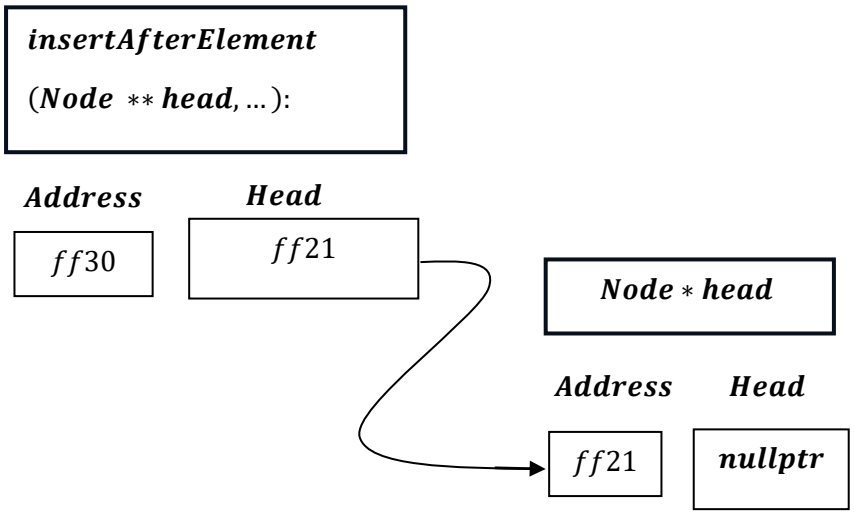
```
void insertAfterElement(Node **head, int data, int after)
{

Node *loc;
loc = searchInUnsortedList(head, after);
.
.
}
….
case 5:
        {
            int data, after;
            cout << "Enter the data: ";
            cin >> data;
            cout << "Enter the element after which to insert: ";
            cin >> after;
            insertAfterElement(&head, data, after);
       }
```

Now, as we call the function :

```
          insertAfterElement(&head, data, after);
```

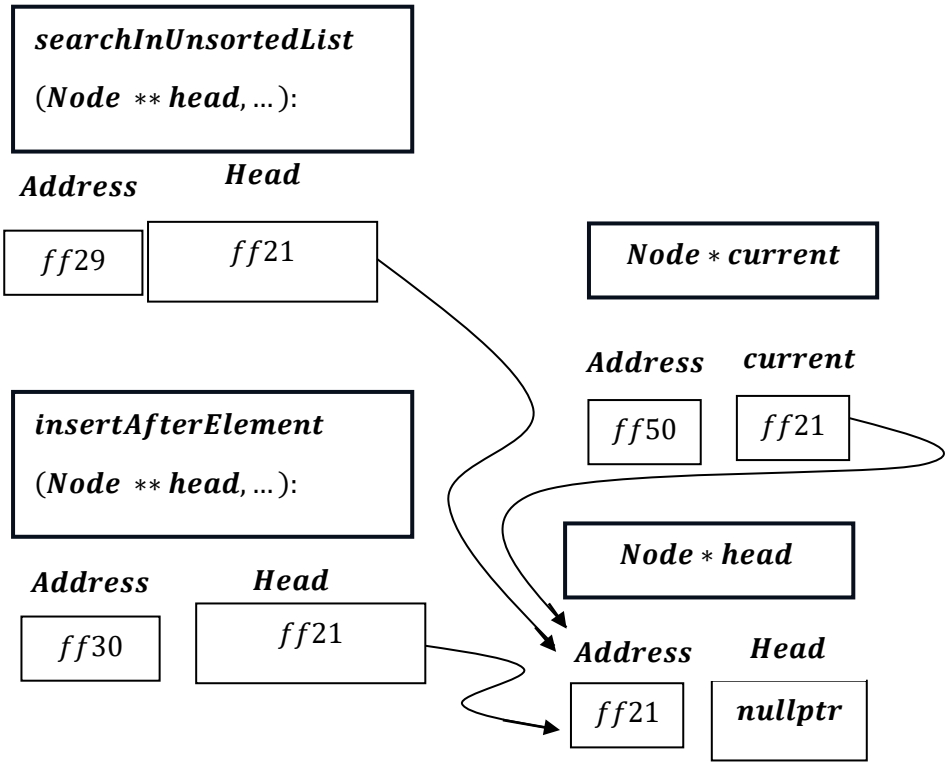
Say, insertAfterElement(&head: ff21, data: 19, after: 21).

But our list is Empty, hence :



And now,

```
Node *loc;
loc = searchInUnsortedList(head, after);
```



*As in function call , we are passing Node ** head's value that have been declared locally in function: searchInUnsortedList function.*

```
loc = searchInUnsortedList(head, after);
```

*Node ** head's value gets initialized to searchInUnsortedList function local Node ** head's value , just like this program:*

```
#include <iostream>
using namespace std;

int main(){

    int **head ;
    int *head_1;
    head = &head_1;
    int **head_2 ;
    head_2 = head;

    int a =10;


    head_1  = &a;


    cout << "Value of a:" << a << endl;

    cout << "Value of a:" << **head << endl;

    cout << "Value of a:" << *head_1 << endl;

    cout << "Value of a:" << **head_2 << endl;

    return 0;
}
```

*Here we see that int ** head_2 is initialized with int ** head.*

And we get `Value of a : 10 ` *everytime of our cout statement.*

*Now, coming back to our program, Node * loc is assigned to `nullptr` as ,*
searchInUnsortedList function returns `nullptr`.

*Node * loc = nullptr*

Node * loc

Address	loc
ff31	nullptr

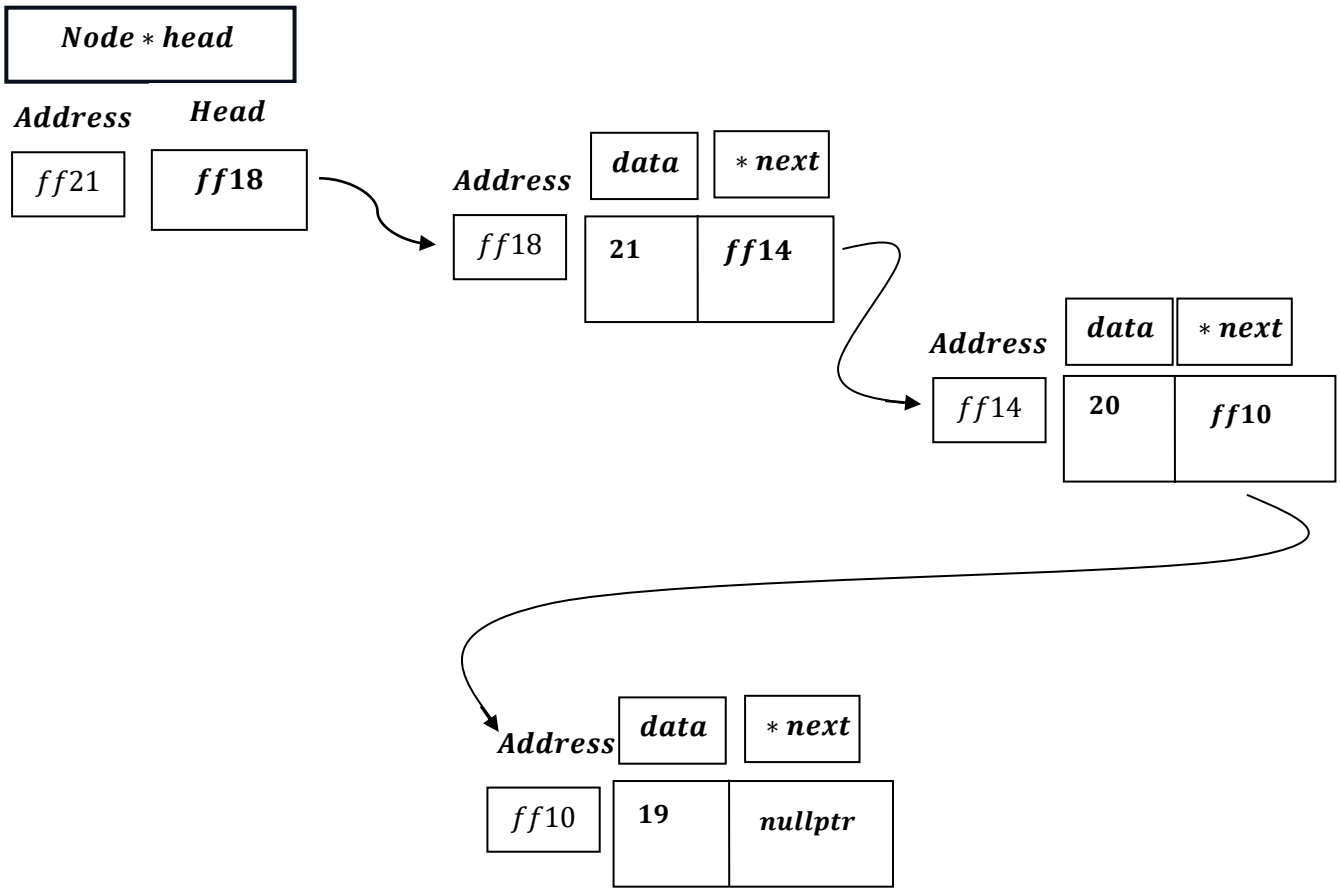
, results:

```
    if (loc == nullptr)
    {
        cout << "Error: Element not found." << endl;
        return;
    }
```

It will print message: "Error: Element not found " and return.

Now, say: $List\ is\ Not\ Empty$

$We\ have\ linked\ list$:

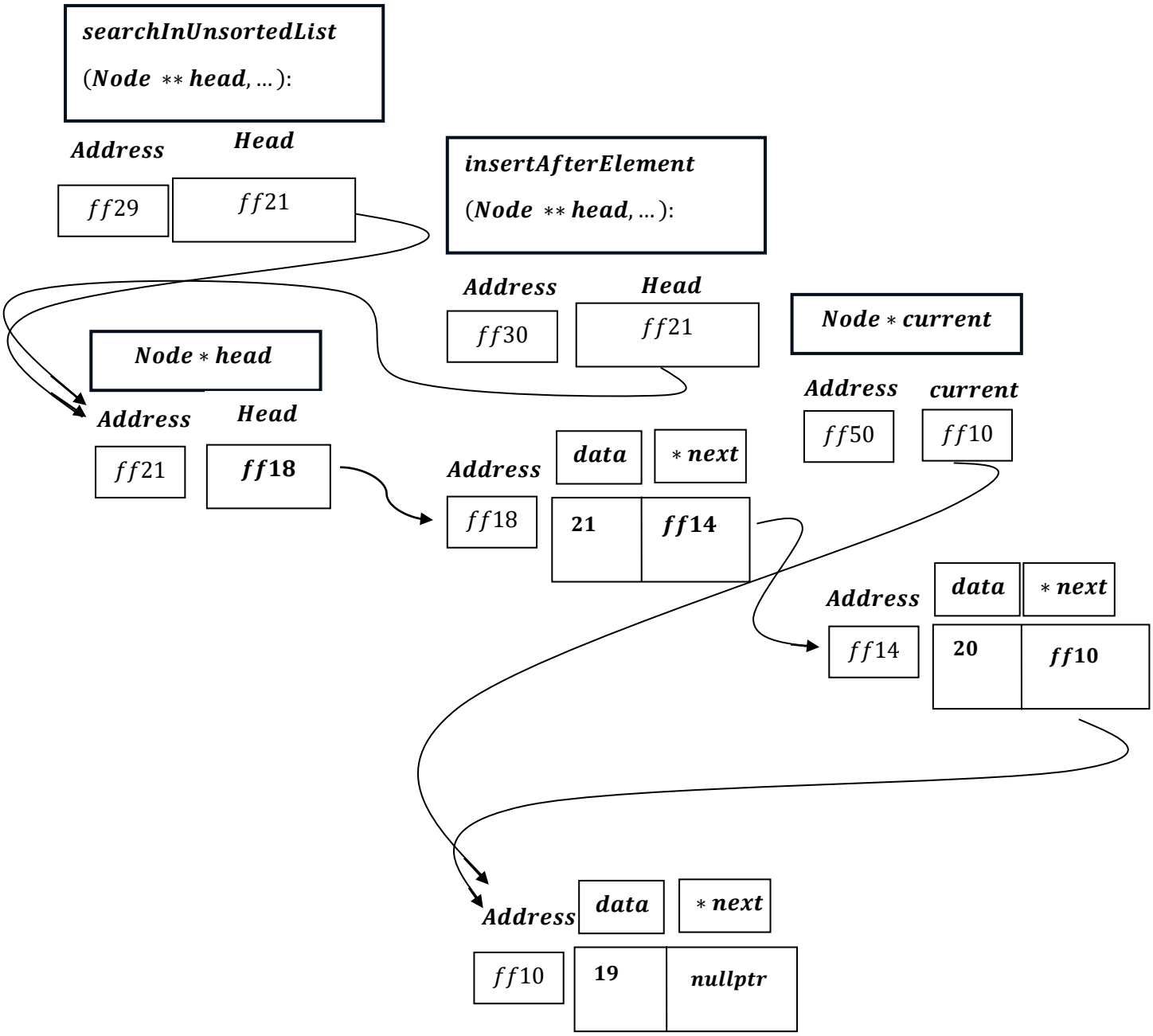


$Say, insertAfterElement(\&head: ff21, data: 18, after: 19)$.

$And\ now,$

```
Node *loc;
loc = searchInUnsortedList(head, after);
```

$function\ call: searchInUnsortedList(\&head: ff21, after: 19)$.

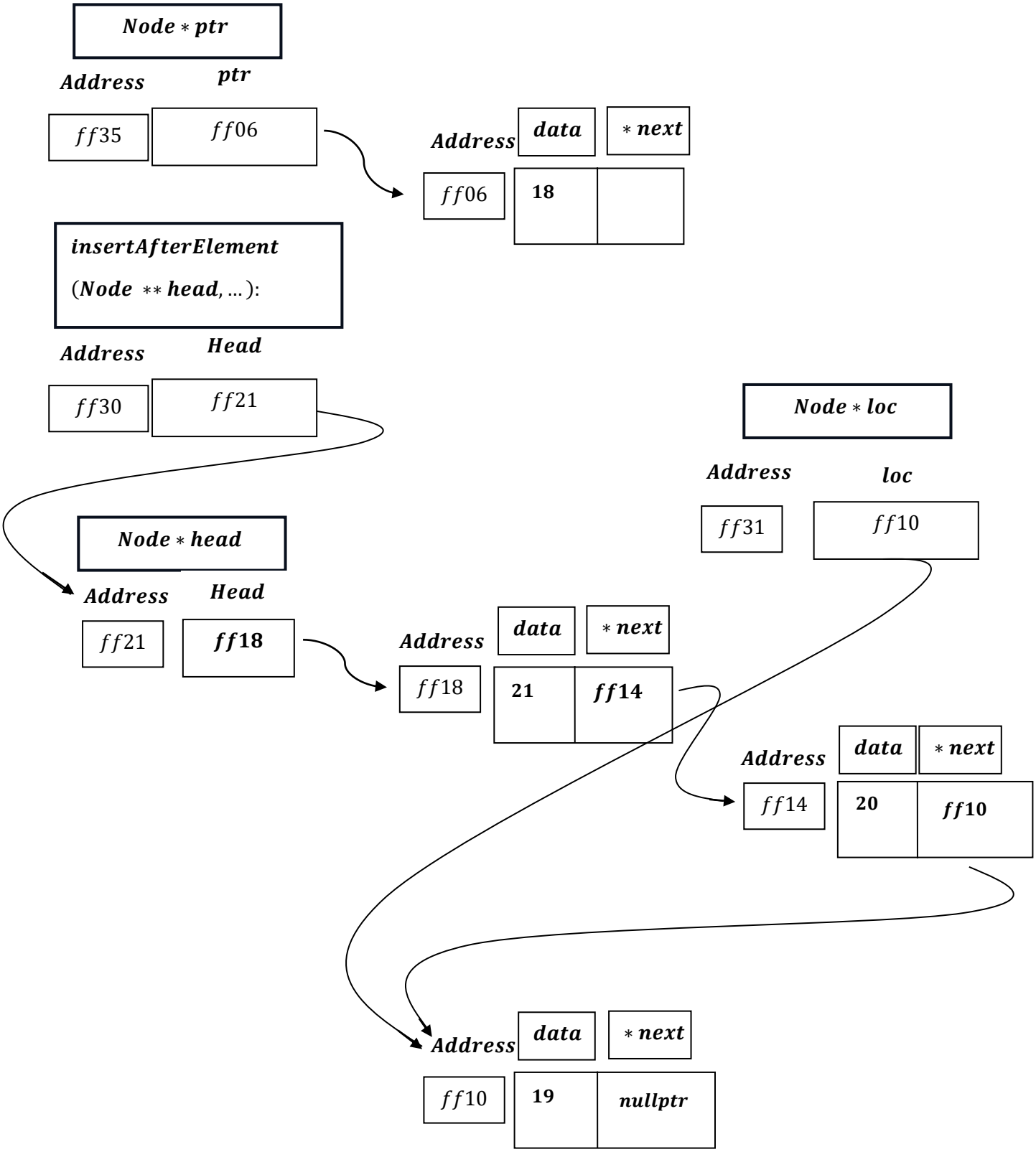


As , $searchInUnsortedList(\&head: ff21, after: 19)$; $returns\ current\ pointer's\ value\ hence$:

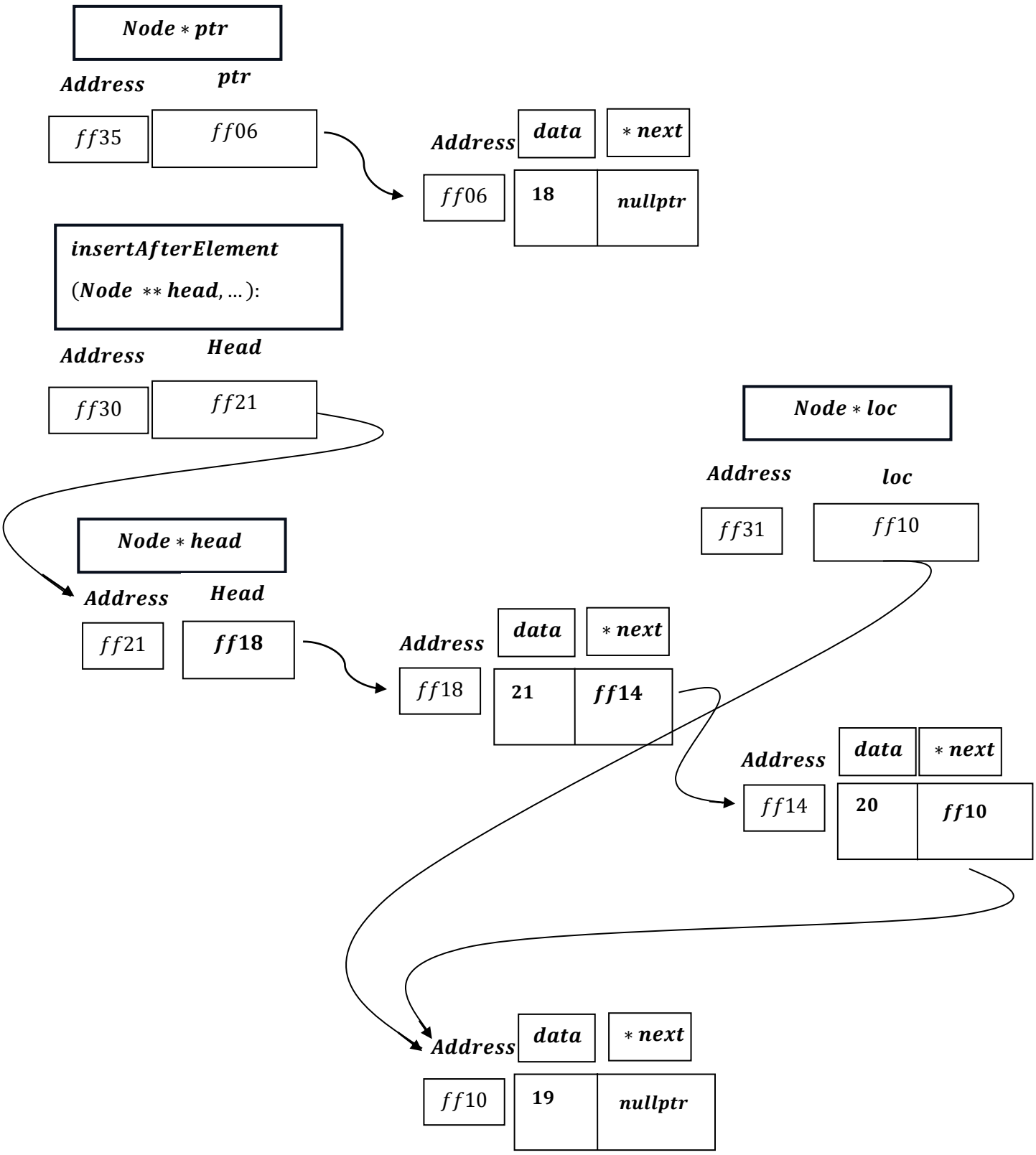
$$Node * loc = ff10;$$

*Now, if checks whether Node * loc equals to `nullptr` which is false, hence:*

```
ptr->data = data;
```

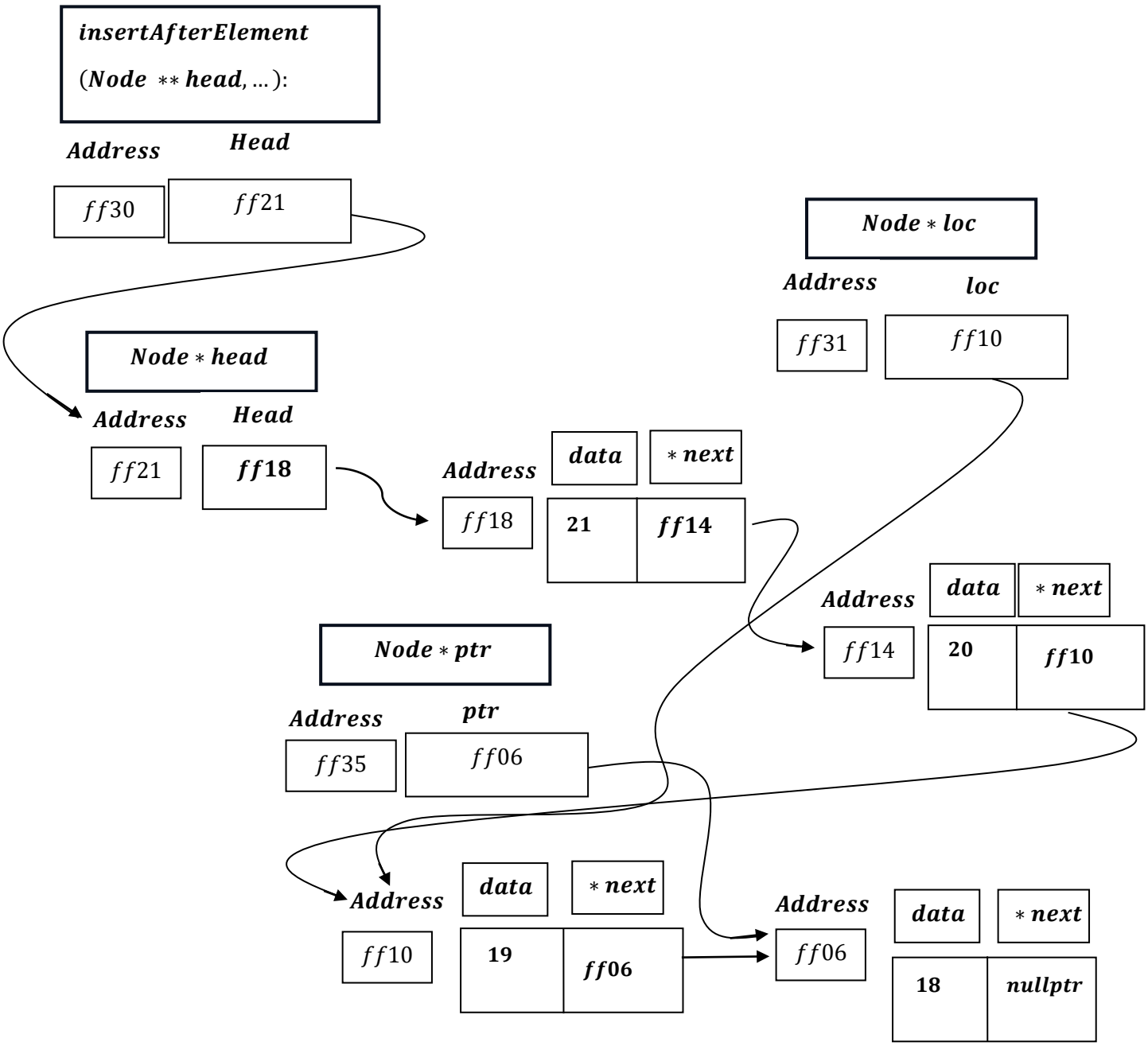


```
ptr->next = loc->next;
```

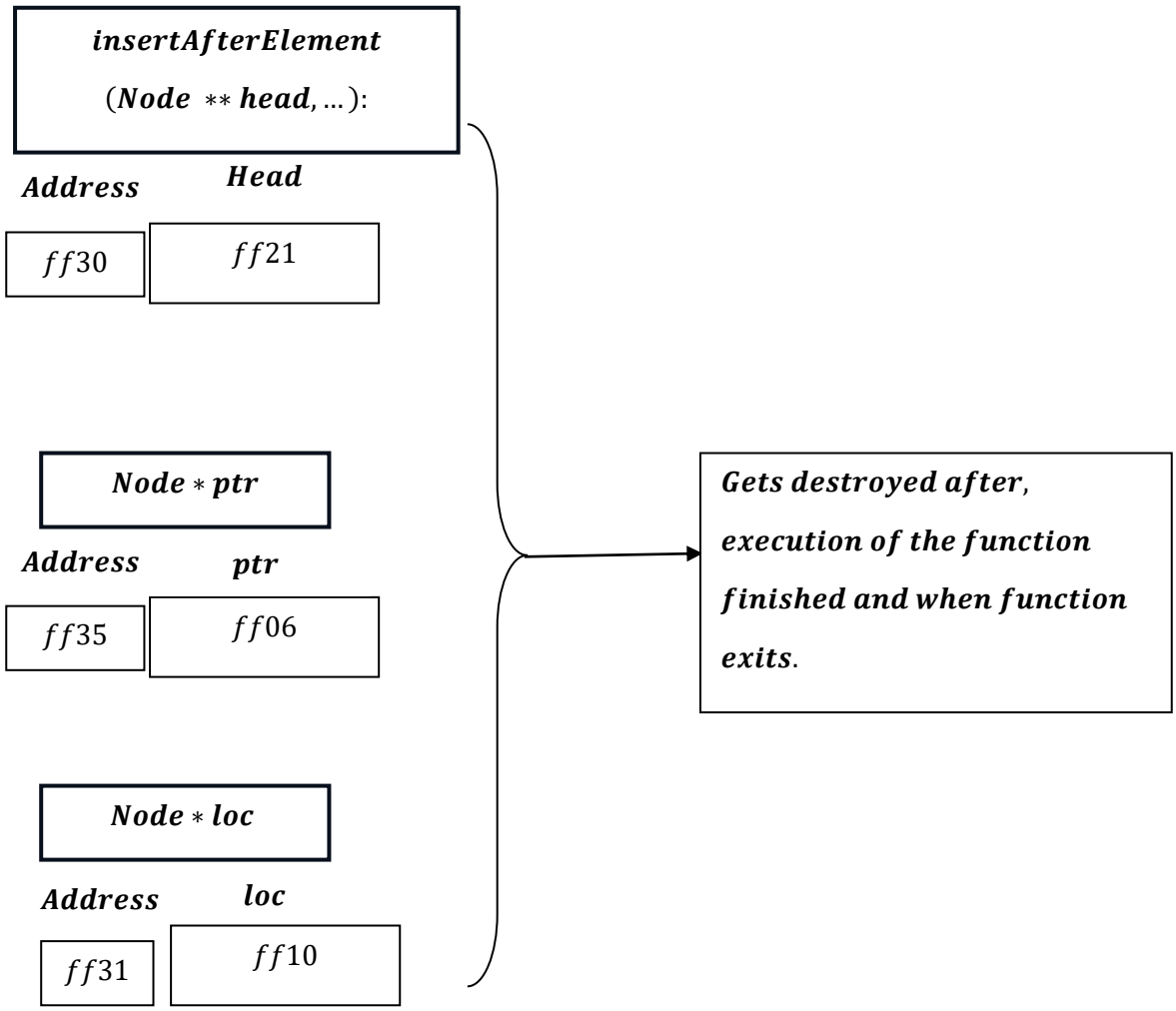


```
    loc->next = ptr;
```

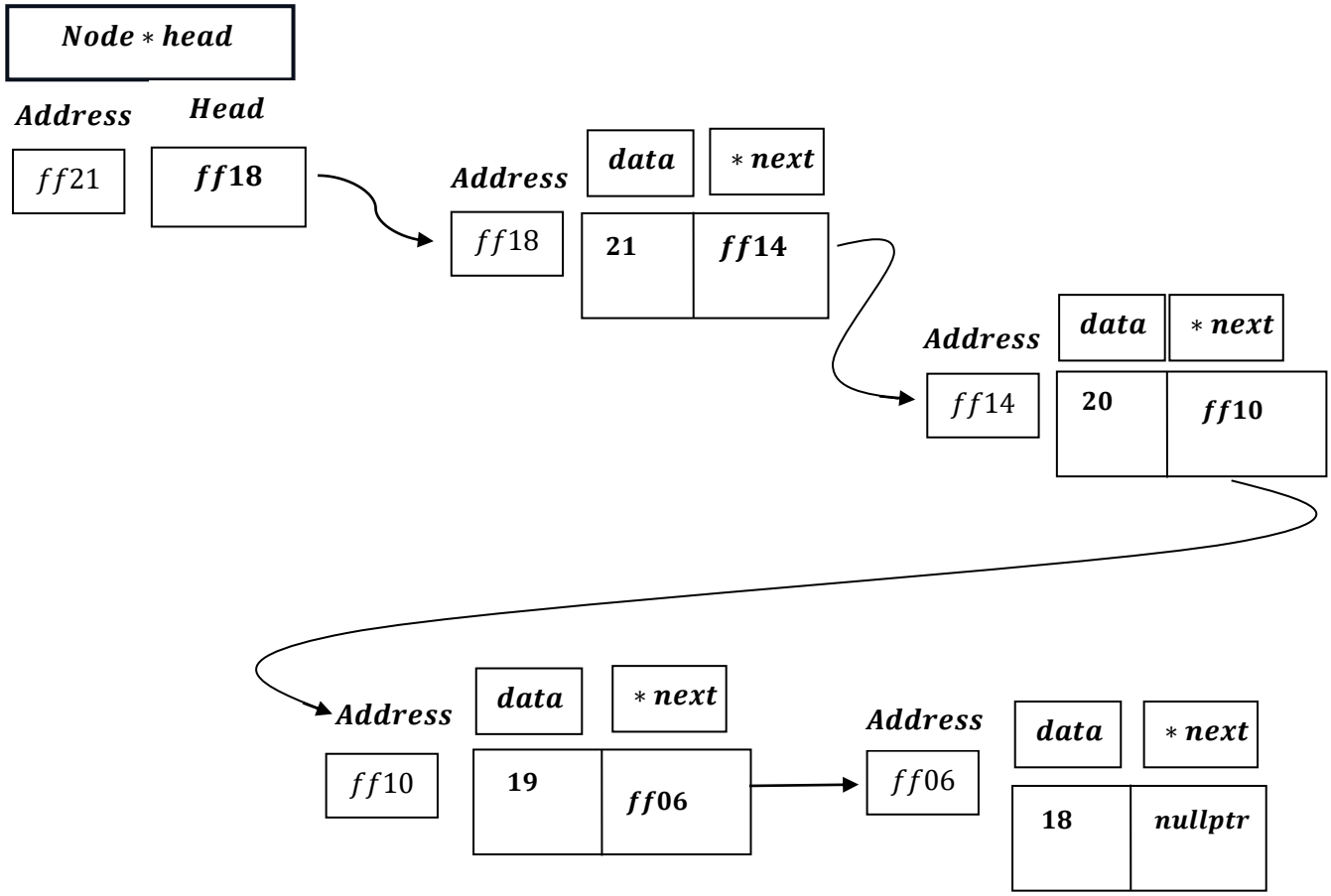
$i.e. loc \rightarrow next = ptr = ff06.$



Now as the function ends , all the temporary pointer variables gets destroyed:



And we are left with:



Now if `key not found`

Say, $insertAfterElement(\&head: ff21, data: 16, after: 17)$.

And now,

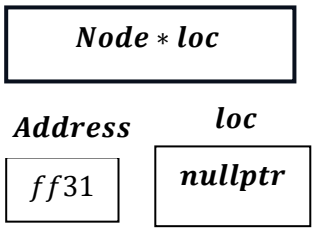
```
Node *loc;
loc = searchInUnsortedList(head, after);
```

function call: $searchInUnsortedList(\&head: ff21, after: 17)$.

As 17 *, not present , then current returns `nullptr` and get assigned*

*Node * loc* .

$Node * loc = nullptr$



,$results$:

```
if (loc == nullptr)
{
    cout << "Error: Element not found." << endl;
    return;
}
```

$It\ will\ print\ message$: "$Error$: $Element\ not\ found$ " $and\ return$.

Time Complexity of
Insert After An Element

```
void insertAfterElement(Node **head, int data, int after)
{
    Node *ptr, *loc;                                  → O(1)
    ptr = (Node *)malloc(sizeof(Node));               → O(1)
    loc = searchInUnsortedList(head, after);          → O(n)
    if (loc == nullptr)
    {
        cout << "Error: Element not found." << endl;   → O(1)
        return;
    }
    ptr->data = data;                                 → O(1)
    ptr->next = loc->next;                            → O(1)
    loc->next = ptr;                                  → O(1)
}
```

<u>Best case (When List is Empty)</u>

→ *Declaration of Node* $*$ *ptr and* $*$ *loc takes constant time i.e.* $O(1)$.

→ *ptr pointer of Node structure allocate memory for member variable data which is of `integer type` and next which is pointer type .Such memory allocation is a fixed amount ,takes constant time complexity i.e.* $O(1)$.

→ *loc pointer of Node structure is assigned to value of current pointer returned by function searchInUnsortedList(head, after). Note here function takes constant amount of time i.e.* $O\ (1)$ *and return current pointer value `nullptr` as list is empty and loc is assigned to that value also takes*

$constant\ amount\ of\ time\ \ i.e.\ O(1).\ Hence\ total\ complexity: O(1) + O(1) = O(1).$

$\rightarrow And\ now, if\ loc\ is\ `nullptr`, checks\ condition, such\ checking\ takes\ constant\ of\ time\ and\ prints\ an\ error\ message: `Error: Element\ not\ found.`\ with\ \ return\ statement. Hence\ as\ whole\ it\ takes\ constant\ amount\ of\ time\ i.e.\ O(1).$

$Hence\ total\ time\ complexity: O(1) + O(1) + O(1) + O(1) = O(1)\ or\ \Omega(1).$

<u>$Worst\ case\ (When\ Key\ not\ Empty)$</u>

$\rightarrow Declaration\ of\ Node * ptr\ and * loc\ takes\ constant\ time\ i.e.\ O(1).$

$\rightarrow ptr\ pointer\ of\ Node\ structure\ allocate\ memory\ for\ member\ variable\ data\ which\ is\ of\ `integer\ type`\ and\ next\ which\ is\ pointer\ type\ . Such\ memory\ allocation\ is\ a\ fixed\ amount\ , takes\ constant\ time\ complexity\ i.e.\ O(1).$

$\rightarrow\ loc\ pointer\ of\ Node\ structure\ is\ assigned\ to\ value\ of\ current\ pointer\ returned\ by\ function\ searchInUnsortedList(head, after). Note\ here\ function\ takes\ `n`\ \ amount\ of\ time\ i.e.\ O\ (n)\ and\ return\ current\ pointer\ value\ `nullptr`\ as\ `Key\ Not\ Found`\ and\ loc\ is\ assigned\ to\ that\ value\ also\ takes\ constant\ amount\ of\ time\ \ i.e.\ O(1). Hence\ total\ complexity: O(n) + O(1) = O(n).$

$\rightarrow And\ now, if\ loc\ is\ `nullptr`, checks\ condition, such\ checking\ takes\ constant\ of\ time\ and\ prints\ an\ error\ message: `Error: Element\ not\ found.`$

$with\ \ return\ statement. Hence\ as\ whole\ it\ takes\ constant\ amount\ of$
$time\ i.e. O(1).$

$Hence\ total\ time\ complexity: O(1) + O(1) + O(n) + O(1) = O(n)$.

$\underline{Average\ Case}$

$(If\ the\ elements\ in\ the\ linked\ list\ are\ uniformly\ distributed$
$and\ the\ key\ is\ equally\ likely\ to\ be\ any\ element\ in\ the\ list)$

$\rightarrow Declaration\ of\ Node * ptr\ and * loc\ takes\ constant\ time\ i.e. O(1).$

$\rightarrow ptr\ pointer\ of\ Node\ structure\ allocate\ memory\ for\ member\ variable$
$data\ which\ is\ of\ `integer\ type`\ and\ next\ which\ is\ pointer\ type\ . Such\ memory$
$allocation\ is\ a\ fixed\ amount\ , takes\ constant\ time\ complexity\ i.e. O(1).$

$\rightarrow\ loc\ pointer\ of\ Node\ structure\ is\ assigned\ to\ value\ of\ current\ pointer$
$returned\ by\ function\ searchInUnsortedList(head, after). Note\ here\ function$
$takes\ \frac{N+1}{2}\ amount\ of\ time\ i.e. O\ (n)\ and\ return\ current\ pointer\ value$
$and\ loc\ is\ assigned\ to\ that\ value\ also\ takes\ constant\ amount\ of\ time$
$i.e. O(1). Hence\ total\ complexity : O(n) + O(1) = O(n).$

$\rightarrow And\ now, if\ loc\ is\ `nullptr`\ , checks\ condition, such\ checking\ takes$
$constant\ of\ time\ and\ prints\ an\ error\ message: `Error: Element\ not\ found.`$

$with\ \ return\ statement. Hence\ as\ whole\ it\ takes\ constant\ amount\ of$
$time\ i.e. O(1).$

$Also\ if\ loc\ is\ not\ `nullptr`\ , checks\ condition\ which\ is\ false, such\ checking\ takes$
$constant\ of\ time\ O(1).$

$\rightarrow ptr\ pointer\ of\ Node\ structure\ pointed\ to\ data\ gets\ assigned\ to$
$data\ (user\ input) takes\ \ constant\ amount\ of\ time\ i.e. O(1).$

$\rightarrow ptr\ pointer\ of Node\ structure\ \ pointed\ to\ next\ pointer\ \ gets\ assigned\ to$
$next\ pointer\ contained\ in\ the\ memory\ location\ and\ that\ location\ is\ pointed$
$by\ loc\ pointer\ of\ Node\ structure\ takes\ constant\ amount\ of\ time\ i.e. O(1).$

$\rightarrow Next\ pointer\ contained\ in\ the\ memory\ location\ and\ that\ location\ is\ pointed$
$by\ loc\ pointer\ of\ Node\ structure\ gets\ assigned\ ptr\ pointer's\ value$
$takes\ constant\ amount\ of\ time\ i.e. O(1)\ .$

$Hence\ total\ time\ complexity: O(1) + O(1) + O(n) + O(1) + O(1) + O(1)$
$= O(n)\ .$

$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get:$
$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$
$\Longrightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N \left[Here\ f(n) = \frac{N+1}{2}\ , because\ of\ Search\ . \right]$

Singly Linked List	***Cases***	***Time Complexity***
Insert After An Element	**1.** ***Best Case***	$\boldsymbol{\Omega(1)}$
	2. ***Worst Case***	$\boldsymbol{O(n)}$
	3. ***Average Case***	$\boldsymbol{\Theta(n)}$

****